
Práctica 1 PMDM: Bitácora Trader

**Programación Multimedia
y Dispositivos Móviles
2.º DAM · MEDAC**

Sergio Mitchell Bocero



Índice

1. Introducción.....	4
2. Parte A. Análisis del desarrollo móvil.....	4
2.1. Ventajas, inconvenientes y limitaciones.....	4
2.2. Elección tecnológica.....	5
2.3. Ciclo de vida de una Activity.....	5
3. Parte B. Especificación de la app.....	6
3.1. Idea y objetivos.....	6
3.2. Requisitos.....	6
3.3. Casos de uso.....	7
4. Parte C. Entorno de desarrollo.....	8
5. Parte D. Recursos de la aplicación.....	9
5.1. Textos e idiomas.....	10
5.2. Imágenes.....	10
5.3. Colores.....	11
5.4. Estilos y tema.....	11
5.5. Icono.....	11
5.6. Ampliación: tema oscuro y horizontal.....	12
6. Conclusión.....	12

Índice de ilustraciones

Ilustración 1. Ciclo de vida de una Activity.....	5
Ilustración 2. Diagrama de casos de uso.....	7
Ilustración 3. Los dos emuladores en Device Manager.....	9
Ilustración 4. La app en el Pixel 8 (español) y en la tablet (inglés, horizontal).....	9
Ilustración 5. Resultado de la inspección de código.....	10
Ilustración 6. Imagen vectorial y mapa de bits.....	10
Ilustración 7. Creación del icono con Image Asset.....	12
Ilustración 8. Tema oscuro y diseño en horizontal.....	12

Índice de tablas

Tabla 1. Limitaciones del desarrollo móvil en Bitácora Trader.....	4
Tabla 2. Comparativa nativo, híbrido y multiplataforma.....	5
Tabla 3. Requisitos funcionales.....	6



Tabla 4. Requisitos no funcionales.....	7
Tabla 5. Ficha del caso de uso CU-01.....	7
Tabla 6. Ficha del caso de uso CU-02.....	8
Tabla 7. Dispositivos virtuales.....	8
Tabla 8. Colores principales de colors.xml.....	11



Bitácora Trader

1. Introducción

En esta práctica he diseñado **Bitácora Trader**, una app Android sencilla para apuntar las operaciones de futuros que hago en una cuenta demo. La idea viene de un trabajo de la universidad en el que opero en simulación y me cuesta acordarme después de por qué entré en cada operación.

No se programa la funcionalidad: el trabajo se centra en el análisis, la especificación, el entorno de desarrollo y los recursos de la app. El proyecto está hecho en Java con vistas XML (Empty Views Activity, API mínima 24) y se entrega en un repositorio público de GitHub.

2. Parte A. Análisis del desarrollo móvil

2.1. Ventajas, inconvenientes y limitaciones

La gran ventaja del móvil es que siempre lo llevas encima, así que puedes apuntar la operación justo al cerrarla. Además, una app nativa funciona sin Internet y va más fluida que una web. Los inconvenientes son que hay menos recursos que en un PC, muchísimos modelos distintos y que hay que publicar en una tienda.

Tabla 1. Limitaciones del desarrollo móvil en Bitácora Trader

Limitación	Cómo afecta a mi app
Batería	La app no hace nada en segundo plano: solo funciona cuando está abierta.
Memoria	Android puede cerrar la app si recibo una llamada; por eso las vistas tienen id y guardan su estado.
Conectividad	Los datos se guardan en el móvil, así que funciona sin conexión.
Tamaño de pantalla	Formulario en una columna, botones grandes y un diseño propio en horizontal.
Fragmentación	minSdk 24, medidas en dp/sp, imágenes vectoriales y prueba en dos emuladores.

2.2. Elección tecnológica

Tabla 2. Comparativa nativo, híbrido y multiplataforma

	Nativo	Híbrido	Multiplataforma
Lenguajes	Java / Kotlin	HTML, CSS, JS	Dart, JS, C#
Rendimiento	El mejor	El más bajo	Casi como nativo
Acceso al hardware	Total	Con plugins	Con plugins
Coste	Alto (una app por sistema)	Bajo	Medio
Ejemplo	Android SDK	Ionic	Flutter

Elijo **nativo Android** porque mis usuarios usan sobre todo Android, porque el sistema de recursos (values-en, values-night, layout-land...) resuelve directamente lo que pide la práctica y porque es lo que estamos viendo en el módulo con Android Studio.

2.3. Ciclo de vida de una Activity

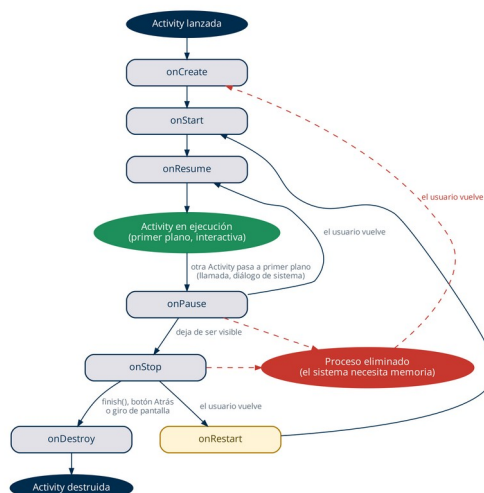


Ilustración 1. Ciclo de vida de una Activity

- **Recibo una llamada:** la llamada pasa delante y se ejecutan `onPause()` y `onStop()`. Al colgar se ejecutan `onRestart()`, `onStart()` y `onResume()` y lo que había escrito sigue ahí.



- **Giro el móvil:** la Activity se destruye (`onPause`, `onStop`, `onDestroy`) y se vuelve a crear (`onCreate`, `onStart`, `onResume`). Al crearse otra vez Android carga el layout de `layout-land`.
- **Pulso Atrás:** en el formulario se llama a `onPause`, `onStop` y `onDestroy` porque la Activity termina, y la pantalla principal vuelve con `onRestart`, `onStart` y `onResume`.

Lo he comprobado poniendo mensajes `Log.d` con la etiqueta «CicloVida» y mirándolos en Logcat.

3. Parte B. Especificación de la app

3.1. Idea y objetivos

Bitácora Trader es un diario de operaciones para quien practica trading de futuros en una cuenta demo. Al cerrar una operación se apunta el instrumento, si fue largo o corto, los precios, los contratos, la estrategia, cómo te sentías y unas notas. En la pantalla de inicio se ve el resumen de la semana. Está pensada para estudiantes y principiantes de 18 a 35 años que aprenden a operar sin arriesgar dinero.

Objetivos medibles:

- Registrar una operación en **menos de 60 segundos** y en un máximo de 2 pantallas.
- Anotar al menos el **90 %** de las operaciones hechas en la cuenta demo durante un mes.
- Ver el resumen semanal **al abrir la app**, sin tener que tocar nada.

3.2. Requisitos

Tabla 3. Requisitos funcionales

Código	Requisito funcional	Prioridad
RF-01	Registrar una operación (instrumento, dirección, precios y contratos).	Alta
RF-02	Calcular el resultado de cada operación en puntos y en dólares.	Alta
RF-03	Mostrar el resumen de la semana en la pantalla de inicio.	Alta
RF-04	Elegir la estrategia y el estado emocional de una lista.	Media

Código	Requisito funcional	Prioridad
RF-05	Consultar el diario y filtrarlo por instrumento.	Media
RF-06	Exportar el diario a CSV.	Baja

Tabla 4. Requisitos no funcionales

Código	Tipo	Requisito no funcional
RNF-01	Usabilidad	Un compañero que no conoce la app registra una operación sin ayuda en menos de 90 s.
RNF-02	Compatibilidad	Funciona desde Android 7.0 (API 24), que cubre casi todos los móviles actuales.
RNF-03	Rendimiento	La pantalla principal carga en menos de 2 s en el emulador.
RNF-04	Idioma	Todos los textos están en strings.xml y traducidos al inglés.

3.3. Casos de uso

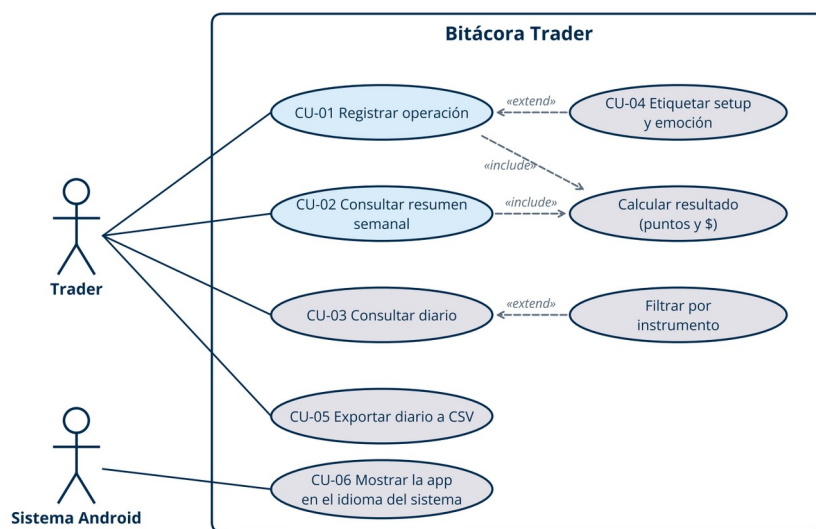


Ilustración 2. Diagrama de casos de uso

Tabla 5. Ficha del caso de uso CU-01

CU-01	Registrar operación
Actor	Trader
Precondición	La app está abierta en la pantalla principal.



CU-01	Registrar operación
Flujo principal	1. Pulsa «Nueva operación». 2. Rellena instrumento, dirección, precios y contratos. 3. Pulsa «Guardar operación». 4. La app guarda la operación y vuelve al inicio.
Flujo alternativo	2a. Falta un precio: la app marca el campo en rojo y no guarda. 3a. Pulsa «Cancelar»: vuelve al inicio sin guardar.
Postcondición	La operación queda guardada y aparece en el resumen.

Tabla 6. Ficha del caso de uso CU-02

CU-02	Consultar resumen semanal
Actor	Trader
Precondición	La app está instalada.
Flujo principal	1. Abre la app. 2. La app calcula operaciones, % de acierto y resultado de la semana. 3. Los muestra en la tarjeta «Resumen de la semana».
Flujo alternativo	2a. No hay operaciones esta semana: muestra 0 y anima a registrar una.
Postcondición	El trader ve su resumen sin modificar datos.

4. Parte C. Entorno de desarrollo

He creado el proyecto `PMDM_P1_SergioMitchell` con la plantilla Empty Views Activity, Java, el paquete `es.medac.sergiomitchell.app` y API mínima 24. Para probarlo he creado dos emuladores distintos:

Tabla 7. Dispositivos virtuales

AVD	Resolución	Densidad	API
Pixel 8 (móvil)	1080 × 2400	420 dpi	35
Medium Tablet	2560 × 1600	320 dpi	35

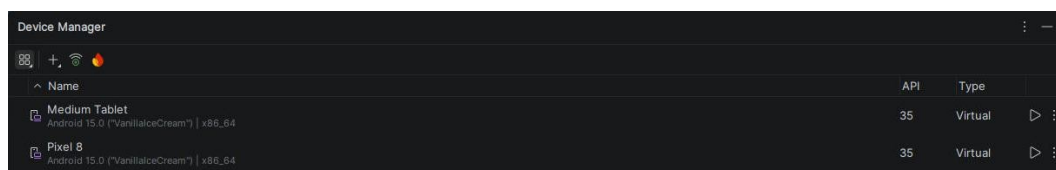


Ilustración 3. Los dos emuladores en Device Manager

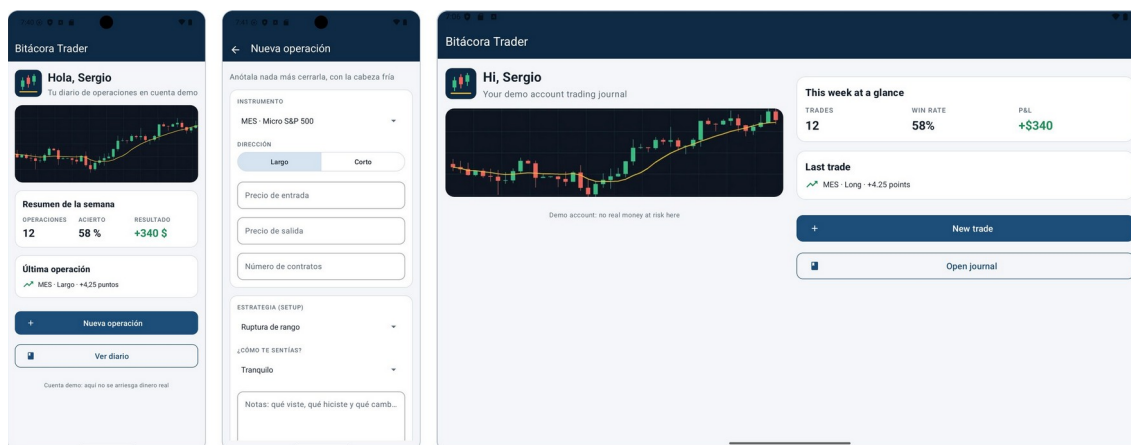


Ilustración 4. La app en el Pixel 8 (español) y en la tablet (inglés, horizontal)

Estructura del proyecto:

- **AndroidManifest.xml**: declara la app, su icono, su nombre, el tema y las Activity.
- **java/**: el código Java (MainActivity y NuevaOperacionActivity).
- **res/**: los recursos (layouts, textos, colores, imágenes...). Las carpetas con sufijo (-en, -night, -land) se usan solas según el idioma, el modo oscuro o la orientación.
- **Gradle**: **settings.gradle** y los **build.gradle** configuran el proyecto, el SDK y las librerías.

La Activity y su layout se unen con `setContentView(R.layout.activity_main)` en `onCreate()`. Después, con `findViewById()` el código accede a cada vista para cambiar textos o responder a clics.

5. Parte D. Recursos de la aplicación

La app tiene dos pantallas: la principal y el formulario «Nueva operación». En los layouts no hay ningún texto, color ni tamaño escrito a mano: todo está en **res/values**.

5.1. Textos e idiomas

`strings.xml` tiene 33 textos traducibles y 3 string-array (instrumentos, estrategias y emociones) que rellenan los desplegables. Hay textos con parámetros, por ejemplo `saludo = «Hola, %1$s»`, que se rellena desde Java con el nombre. La traducción completa está en `values-en/strings.xml`: con el emulador en inglés la app sale en inglés (la tablet de la ilustración anterior) y en español en el resto de casos.

La inspección de Lint no muestra ningún aviso de «Hardcoded text»:

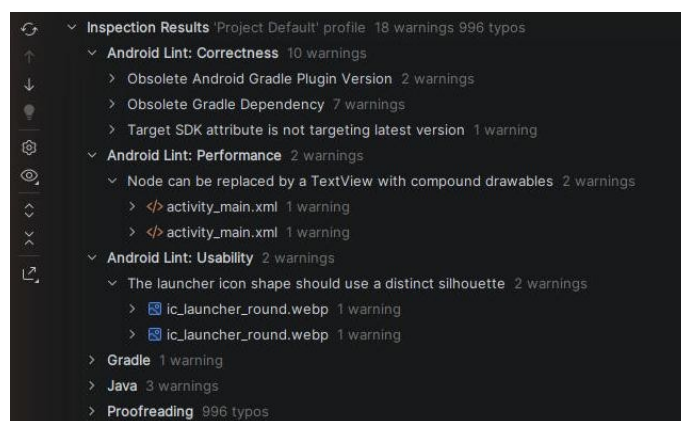


Ilustración 5. Resultado de la inspección de código

5.2. Imágenes



ic_logo_bitacora.xml
Vectorial · 1,4 KB



img_grafico_velas.webp
Mapa de bits · 720×270 px · 8,6 KB

Ilustración 6. Imagen vectorial y mapa de bits

El **logo y los iconos son vectoriales** (Vector Asset): ocupan muy poco y se ven bien en cualquier pantalla. El **gráfico de velas es un mapa de bits** en WebP (8,6 KB), porque tiene mucho detalle. Los vectores son mejores para iconos y logos porque reducen el tamaño de la

app; los mapas de bits son para fotos o imágenes complejas, y conviene comprimirlos (WebP) para que la app no pese.

5.3. Colores

Tabla 8. Colores principales de `colors.xml`

Nombre	Color	Uso
<code>color_barra_superior</code>	#0F2A44	Barra superior y fondo del icono
<code>color_primario</code>	#1D4E7A	Botón principal y bordes
<code>color_acento</code>	#F2A900	Detalles que deben destacar
<code>color_ganancia</code>	#1E8E5A	Resultados positivos
<code>color_perdida</code>	#C8372D	Resultados negativos y errores
<code>color_fondo</code>	#F4F6F9	Fondo de las pantallas
<code>color_texto_principal</code>	#14202E	Títulos y datos

He elegido **azul marino** porque transmite seriedad, como las plataformas de trading; **verde y rojo** porque son los colores de las velas (sube y baja), y **ámbar** como acento. Los nombres indican la función del color, no el tono.

5.4. Estilos y tema

En `styles.xml` hay estilos propios que se reutilizan en las dos pantallas, por ejemplo `Estilo.Titulo`, `Estilo.Etiqueta`, `Estilo.Boton.Principal`, `Estilo.Boton.Secundario` y `Estilo.Tarjeta`. El tema `Theme.BitacoraTrader` hereda de Material 3 y usa mis colores. Las medidas están en `dimens.xml`, con **dp** para tamaños y márgenes y **sp** para los textos.

5.5. Icono

El icono lo he creado con **Image Asset** (Launcher Icons, Adaptive and Legacy) a partir de dos SVG propios: un fondo azul marino y tres velas con una línea ámbar. Image Asset genera las carpetas `mipmap-*` y el icono adaptativo, y en el Manifest se aplica con `android:icon="@mipmap/ic_launcher"`.

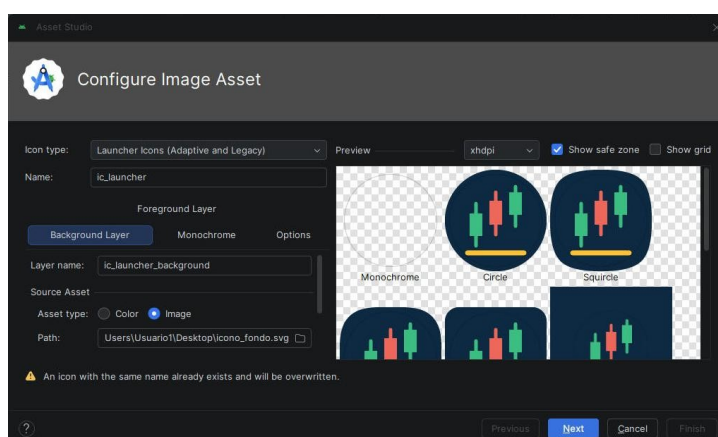


Ilustración 7. Creación del icono con Image Asset

5.6. Ampliación: tema oscuro y horizontal

He hecho las dos ampliaciones: **values-night** con los colores para el modo oscuro y **layout-land** con la pantalla principal a dos columnas.



Ilustración 8. Tema oscuro y diseño en horizontal

6. Conclusión

He aprendido que en Android los recursos se organizan antes de programar. Al principio me parecía raro no poder escribir textos en el layout, pero luego traducir la app fue tan fácil como crear un fichero, y el modo oscuro igual. También entendí mejor el ciclo de vida viendo los mensajes en Logcat al girar el móvil. Las dificultades fueron que los emuladores tardaban mucho en arrancar en mi ordenador, que en inglés el resultado salía mal colocado («\$+340») y tuve que cambiar el texto con parámetros, y que el botón seleccionado salía en un color que no era de mi paleta hasta que lo añadí al tema.